Fold DB: Preventing Data Structure Fragmentation Through Vector Embedding-Based Canonical Schema and Field Resolution

Tom Tang Fei Jia

March 12, 2026

Abstract

When schemas are independently created from multiple sources, fragmentation occurs along two axes: *schema-level*, where the same entity is represented by near-duplicate schemas with different names; and *field-level*, where semantically identical fields appear under different names (e.g., "artist" vs. "composer"), preventing cross-schema queries. We introduce a two-tier canonicalization system using vector embeddings to resolve both forms of fragmentation. At the field tier, each field carries a text description that is embedded with schema context and matched against a canonical field registry using cosine similarity with bidirectional verification. At the schema tier, descriptive names are embedded and compared for semantic similarity, with an identity hash (SHA-256 of sorted canonical field names) providing exact structural deduplication. Field-level canonicalization runs first, causing the identity hash to converge before schema-level deduplication. When a candidate schema matches an existing schema but adds new fields, the system creates an expanded schema whose shared fields carry *field mappers*—pointers to the now-deprecated original schema's fields—inheriting molecule UUIDs. The deprecated schema is blocked; all subsequent writes flow through the expanded schema, adding data directly on top of existing molecules. No data migration occurs. The result is a self-organizing namespace that converges toward canonical representations as more data is ingested, without requiring manual curation or centralized ontologies.

1 Introduction

When a database accepts schemas from multiple sources—users designing schemas manually, AI systems inferring schemas from data, import tools converting external formats, or applications generating schemas programmatically—the independence of each schema creation event produces systematic fragmentation along two axes that compound each other.

Schema-level fragmentation. The same conceptual entity is represented by multiple schemas. A user ingests contact information from their email client, their phone's address book, and a CRM export. Each source produces its own schema: "Email Contacts," "Phone Address Book," and "CRM Contacts." All three describe the same entity (a person with name, email, phone number) but exist as separate schemas. Data cannot be queried across them. A search for "all contacts named Smith" requires three separate queries and manual result merging.

Field-level fragmentation. Semantically identical fields appear under different names. Even if a system detects that "Email Contacts" and "Phone Address Book" describe the same entity

and merges them, the merged schema may contain both `email_address` (from the email client) and `e_mail` (from the phone) as separate fields, each pointing to separate data molecules. The structural duplication has been resolved, but the semantic duplication persists.

These two forms of fragmentation compound: schema-level fragmentation creates many schemas with overlapping purposes, and field-level fragmentation ensures that even when schemas are merged, the merged result contains redundant fields. Neither form can be fully resolved without addressing the other.

This paper makes three contributions:

1. A *two-tier canonicalization system* that resolves both schema-level and field-level fragmentation using vector embeddings, with field-level resolution feeding into schema-level deduplication through identity hash recomputation.

2. A *description-based matching* approach where canonical field resolution operates on text descriptions of fields rather than field names, enabling synonym resolution and homonym disambiguation without manual curation.

3. A *zero-migration schema expansion* mechanism using field mappers to deprecated schemas, where expanded schemas inherit molecule UUIDs by reference and accumulate data on top of existing molecules with no data copying at any step.

1.1 Motivating Example

Consider a personal database that ingests data about musical works from three sources:

- **Spotify export**: schema "Music Library," fields `artist`, `album_title`, `release_year`, `genre`
- **Vinyl catalog scan**: schema "Record Collection," fields `performer`, `record_name`, `year`, `style`
- **Concert program PDF**: schema "Concert Programs," fields `composer`, `work_title`, `premiere_date`, `musical_form`

Without any deduplication, the database contains 3 schemas and 12 fields describing 4 concepts across 4 semantic categories (creator, title, date, genre). Every query about "music" must be manually directed to the correct schema, and aggregation across sources is impossible.

Schema-level deduplication alone might merge these into one or two schemas, but the merged result still contains `artist`, `performer`, and `composer` as three separate fields for the same concept, each storing data in separate molecules.

Field-level deduplication alone might canonicalize field names within each schema, but the three schemas remain independent structures that cannot be queried together.

Only the combination of both—recognizing that these schemas describe the same entity AND that their fields refer to the same concepts—produces a unified data structure where one schema with canonical fields serves all three data sources (Figure 4).

1.2 Related Work

ETL pipelines are manually authored per source-target pair and do not generalize. **Schema-on-read** (data lakes) defers fragmentation to query time. **Naming conventions** require prescribed vocabularies impractical across diverse sources. **Field dictionaries/ontologies** (OWL, RDF)

require expert curation and do not scale. **Whole-schema embedding similarity** (prior work by the present authors) prevents schema-level proliferation but does not resolve field-level synonymy. **Identity hashing** (SHA-256 of sorted field names) provides exact deduplication but is purely syntactic.

None of these operate at both schema and field granularity with semantic similarity, content-addressed storage integration, and zero manual curation.

2 System Architecture

The two-tier canonicalization system integrates into the schema registration pipeline, processing each candidate schema before it is committed to the registry (Figure 1). The system is agnostic to how the candidate schema was created. It comprises six components:

(a) Schema embedder. Converts schema descriptive names into dense vector representations for schema-level similarity comparison.

(b) Schema registry. Stores all registered schemas with their identity hashes, descriptive name embeddings, field definitions, and field-to-molecule mappings. Supports three lookup modes: exact identity hash match, semantic embedding similarity, and field-overlap analysis.

(c) Field embedder. Converts field text descriptions with schema context into dense vector representations for field-level similarity comparison.

(d) Canonical field registry. Stores all canonical field identities with their embeddings, aliases, and usage counts.

(e) Resolution engine. Orchestrates the two-tier resolution process: field-level canonicalization first, then schema-level deduplication.

(f) Mutation mapper generator. Produces rename mappings for both schema names and field names, enabling transparent data redirection during mutation generation.

3 Tier 1: Field-Level Canonical Resolution

The first stage of the pipeline resolves individual fields against the canonical field registry (Figure 2). This tier runs before schema-level deduplication so that the identity hash comparison operates on canonicalized field names. Every field in every candidate schema passes through this resolution, regardless of whether the schema will later match an existing schema or register as new.

3.1 Field Descriptions as the Unit of Matching

Canonical field matching operates on **text descriptions** of fields, not on field names. This is a critical design choice: field names are programmer-chosen identifiers that are unreliable signals of semantic intent. Two fields named `title` may describe entirely different concepts (a book's title vs. an employee's job title), while two fields named `artist` and `composer` may describe the same concept (the person who created a musical work).

Each field carries a text description that captures its semantic meaning, provided at schema creation time—by a human designer, an AI inference system, an import tool that extracts descriptions from source metadata, or any process capable of annotating fields with natural-language semantics. Examples:

- Field `artist`: "the person or group who performed or recorded the musical work"
- Field `composer`: "the person who wrote or composed the musical piece"
- Field `title` (in a book schema): "the name of the published book or written work"
- Field `title` (in an HR schema): "the employee's job title or position within the organization"

These descriptions are the input to the embedding model, not the field names. The embedding function is:

$$\text{embed}(d, s) = E(\text{“}d\text{, in the context of } s\text{”})$$

where d is the field's text description and s is the schema's descriptive name. The schema context disambiguates descriptions that might be similar in isolation: "the person who created the work, in the context of a Music Library" vs. "the person who created the work, in the context of a Patent Database" may produce different canonical resolutions because "creator" means different things in different domains.

Description-based matching has three advantages over name-based matching:

Synonym resolution. Fields with different names but the same description ("the person who created the musical work") resolve to the same canonical field regardless of whether the schema creator chose `artist`, `performer`, or `composer`.

Homonym disambiguation. Fields with the same name but different descriptions ("the name of the book" vs. "the employee's job position") resolve to different canonical fields, preventing false unification.

Richer semantic signal. A description like "the total amount charged to the customer including tax and shipping" carries far more semantic information than the field name `total_cost`, enabling more accurate similarity matching.

3.2 Nearest Canonical Neighbor Search

For each field f_i with description d_i in the candidate schema with descriptive name s, compute the context-enriched embedding $\mathbf{v}_i = \text{embed}(d_i, s)$ and find the canonical field with the highest cosine similarity:

$$c^* = \arg\max_{c \in \mathcal{C}} \text{sim}(\mathbf{v}_i, \mathbf{e}_c)$$

where $\mathcal{C}$ is the set of all canonical fields and $\mathbf{e}_c$ is the embedding of canonical field c's description.

If $\text{sim}(\mathbf{v}_i, \mathbf{e}_{c^*}) < \tau_{\text{field}}$ (default 0.88), the field's description does not match any canonical field. A new canonical field is created with f_i as its name, d_i as its canonical description, and $\mathbf{v}_i$ as its embedding.

3.3 Bidirectional Verification

If $\text{sim}(\mathbf{v}_i, \mathbf{e}_{c^*}) \geq \tau_{\text{field}}$, the system performs a reverse check: among all candidate fields F, which field is most similar to c^*?

$$f^* = \arg\max_{f_j \in F} \text{sim}(\mathbf{v}_j, \mathbf{e}_{c^*})$$

The match is accepted only if $f^* = f_i$. This bidirectional constraint prevents false positives from asymmetric similarity distributions.

Example. A schema with fields `medium` ("the material or technique used to create the artwork") and `creator` ("the person who made the artwork"). The canonical registry contains `artist` with description "the person who created or performed the work." The description embedding of `medium` has moderate similarity (0.86) to `artist`'s description because both relate to art creation. But `creator`'s description has higher similarity (0.93) because it directly describes the same concept. The forward pass would match `medium` to `artist`, but the reverse pass reveals `creator`'s description is a better match. The bidirectional check rejects `medium`→`artist` and accepts `creator`→`artist`.

3.4 Canonical Field Establishment and Update

When a field's description does not match any canonical field (similarity below threshold or bidirectional verification fails), a new canonical field is established with the field's name as the canonical name, the field's description as the canonical description, and its description embedding as the canonical embedding.

When a field's description matches an existing canonical field's description, the system:

1. Renames the field to the canonical field's name.
2. Adds the original field name and description to the canonical field's alias set (e.g., canonical field `artist` with description "the person who created or performed the work" accumulates aliases: `creator`/"the person who made the artwork," `performer`/"the musician or band who played the song," `composer`/"the person who wrote the musical composition").
3. Increments the usage count.
4. Optionally updates the canonical embedding as a weighted centroid:

$$\mathbf{e}'_c = \frac{n \cdot \mathbf{e}_c + \mathbf{v}_i}{n+1}$$

   where n is the usage count. The update is bounded: if the updated embedding diverges from any alias's embedding by more than $(1 - \tau_{\text{field}})$, the update is rejected to prevent drift.

3.5 Identity Hash Recomputation

After canonical field resolution renames fields, the schema's identity hash is recomputed:

$$h' = \text{SHA256}\big(\text{join}(\text{sort}(f'_1, f'_2, \ldots, f'_n), \text{“,”})\big)$$

where f'_i are the canonicalized field names. This is the critical bridge between the two tiers: schemas that were structurally distinct before canonicalization (because they used synonyms like `artist` vs. `performer`) now produce the same identity hash, enabling exact-match deduplication to catch them.

4 Tier 2: Schema-Level Deduplication

After field-level canonicalization produces canonical field names and a recomputed identity hash, the system performs schema-level deduplication in three sequential passes (Figure 3).

4.1 Exact Identity Hash Match

The identity hash h' (as recomputed in Section **??**) is looked up in the schema registry. If found, the candidate schema has an identical canonicalized field set to an existing schema. The existing schema is returned immediately—no further processing is needed. This provides $O(1)$ exact-match lookup regardless of registry size.

4.2 Descriptive Name Embedding Similarity

If no exact hash match is found, the system embeds the candidate schema's descriptive name and compares it against all registered descriptive name embeddings:

$$s^* = \arg\max_{s_j \in \mathcal{S}} \operatorname{sim}(E(s), E(s_j))$$

where $\mathcal{S}$ is the set of all registered schemas and E is the embedding model.

If $\operatorname{sim}(E(s), E(s^*)) \geq \tau_{\text{schema}}$ (default 0.80), the candidate is identified as a semantic duplicate or expansion candidate for schema s^*. This catches cases where two schemas describe the same entity under different names (e.g., "Music Library" and "Song Collection").

Schema-level matching uses the short descriptive name (a phrase like "Music Library") rather than a richer text description. This is sufficient because schema names are purpose-built labels that carry adequate semantic signal, and because the field-level tier provides the fine-grained semantic resolution. The schema tier's role is coarser: determining whether two schemas describe the same *category* of entity. The lower similarity threshold ($\tau_{\text{schema}} = 0.80$ vs. $\tau_{\text{field}} = 0.88$) reflects this coarser granularity.

4.3 Field Overlap Analysis

If no descriptive name match is found, the system performs a field overlap check against all registered schemas. For each registered schema R with field set F_R:

$$\operatorname{overlap}(F, F_R) = |F \cap F_R|$$

If $\operatorname{overlap}(F, F_R) \times 2 > \min(|F|, |F_R|)$—that is, the overlapping fields constitute more than half of the smaller schema's fields—the candidate is identified as an expansion candidate. This catches cases where descriptive names differ significantly but the field structures reveal shared purpose.

After canonical field resolution (Tier 1), this overlap check becomes more powerful: fields that were syntactically different but semantically identical now share the same canonical name, increasing the measured overlap.

4.4 Schema Expansion via Field Mappers to Deprecated Schemas

When a semantic match or field overlap is detected and the candidate schema contains fields not present in the matched schema, the system creates an **expanded schema** with **field mappers** pointing back to the now-deprecated original schema. This is the central mechanism by which the system grows data structures over time with **zero data migration**:

1. The expanded schema's field set is the union of the existing and candidate field sets.
2. For each shared field, the expanded schema establishes a **field mapper**: a pointer from the expanded schema's field to the corresponding field in the existing (now deprecated) schema. This pointer causes the expanded field to *inherit* the existing field's molecule UUID. The molecule is not copied—the expanded schema's field now points directly to the same molecule that the deprecated schema created and wrote to.
3. The existing schema is set to the "Blocked" state, preventing further writes through it. All data previously written through the deprecated schema remains intact: it lives in the molecules, and the expanded schema's field mappers provide direct access to those molecules.
4. New writes through the expanded schema go to the inherited molecules for shared fields, **adding new data directly on top of data that was previously written through the deprecated schema**. Data written through the deprecated schema and data written through the expanded schema are indistinguishable—the expanded schema's field mapper points to the exact same molecule that the deprecated schema pointed to, and all writes go to that single molecule regardless of which schema was active at the time.
5. New fields (present in the candidate but not the deprecated schema) receive fresh molecule UUIDs and begin accumulating data from this point forward.
6. The expanded schema's identity hash is recomputed from the merged, canonicalized field set.

This design has three critical properties:

Property 1 (Zero Migration)**.** *No data is copied, moved, or rewritten during expansion. Field mappers transfer molecule ownership by reference, not by value. A database with millions of records in the deprecated schema's molecules pays zero I/O cost during expansion.*

Property 2 (Continuous Accumulation)**.** *New data written through the expanded schema accumulates in the same molecules that the deprecated schema populated. The expanded schema is not a "fresh start"—it is a structural evolution that inherits all prior data. A query for "all records" returns both pre-expansion and post-expansion data from the same molecule.*

Property 3 (Iterative Growth)**.** *Schema expansion composes: a schema that starts with 4 fields from Source A can expand to 6 fields when Source B adds 2 new fields, then to 8 fields when Source C adds 2 more. At each expansion, the previous schema is deprecated, field mappers inherit its molecules, and new fields get fresh molecules. The chain of expansions ($A \to A{+}B \to A{+}B{+}C$) produces a single live schema whose molecules contain all data from all sources, with no migration at any step.*

5 Canonical Field Registry

The canonical field registry maintains the set of all canonical fields recognized by the system. Each entry contains:

- **Canonical name**: the authoritative field name for this concept (e.g., `artist`).
- **Canonical description**: the authoritative text description of what this field represents. This is the primary identity of the canonical field—matching is performed against this description, not the name.

- **Canonical embedding**: the context-enriched embedding vector of the canonical description, updated as a bounded centroid.
- **Aliases**: the set of alternative field names and descriptions that have resolved to this canonical field, preserving provenance.
- **Schema contexts**: the set of schema descriptive names in which this canonical field appears, used for homonym disambiguation.
- **Usage count**: the number of schemas referencing this canonical field, used for ranking.

The registry grows organically. When the first schema in a domain is registered, its field descriptions establish the initial canonical fields. As subsequent schemas arrive—regardless of how they were created—their field descriptions are compared against existing canonical descriptions. Over time, the registry converges toward a stable set of canonical field identities for each domain, with the canonical description serving as the semantic anchor that field names alone cannot provide.

6 Mutation Mappers

When either tier renames a schema or field, a mutation mapper is produced: a mapping from the original name to the canonical name. During mutation generation, the mapper is applied so that incoming data using non-canonical names is transparently written to canonical structures.

Mutation mappers compose across tiers. If a field `performer` resolves to canonical field `artist`, and the schema "Record Collection" resolves to "Music Library," the composed mapper redirects writes of `performer` in "Record Collection" to `artist` in "Music Library"—a single transparent redirection that handles both levels of canonicalization.

7 Integration with Molecule-Centric Storage

In Fold DB, each schema field points to a content-addressed data molecule (UUID-identified). The two-tier canonicalization system produces molecule-level data sharing through schema expansion. When expansion occurs (Section **??**), field mappers inherit molecule UUIDs by reference. The deprecated schema's molecules accumulate data continuously across schema versions—from the database's perspective, the expansion is invisible.

Example. Source A creates S_1 (4 fields, 10,000 records). Source B triggers expansion to S_2 (6 fields): S_1 is deprecated, S_2's shared fields inherit S_1's molecules, 2 new fields get fresh molecules. Source C triggers expansion to S_3 (8 fields): same process. At the end, S_3 is the single live schema containing all data from all sources, and **no data was ever migrated**.

8 Retroactive Canonicalization

When a new canonical match subsumes previously independent structures, the system scans all registered schemas for affected fields, recomputes identity hashes, and proposes unification. For molecule merging, the default strategy selects the molecule with the highest version count; conflicts from concurrent writes to formerly independent molecules are flagged for manual resolution. Retroactive canonicalization is non-destructive: unification is proposed, not imposed.

9 Edge Cases and Thresholds

9.1 Homonym Handling

Context-enriched embedding naturally disambiguates most homonyms, but edge cases remain:

Schema-domain clustering. When a candidate field's schema context is semantically distant from all existing contexts for a canonical field, the effective similarity score is reduced by $\lambda \cdot d_{\text{domain}}$ (default $\lambda = 0.05$), penalizing cross-domain matches.

Coherence monitoring. If a canonical field's alias set exceeds a threshold (default 20) and average pairwise similarity among alias embeddings drops below $\tau_{\text{field}} - 0.05$, the canonical field is flagged for splitting via k-means clustering on alias embeddings.

9.2 Threshold Selection

Schema threshold ($\tau_{\text{schema}} = 0.80$). Lower because schema names are short phrases with less precise embeddings. Catches "Music Library" $\approx$ "Song Collection" while distinguishing "Music Library" from "Video Library."

Field threshold ($\tau_{\text{field}} = 0.88$). Higher because field descriptions are more semantically specific. Preliminary evaluation suggests $\sim$3% false positive rate with bidirectional verification at 0.88 (vs. $\sim$12% at 0.85, <1% at 0.92 with $\sim$15% false negatives). Domain-specific overrides are supported.

10 Limitations

Serial processing bottleneck. Each candidate schema must complete full resolution before the next can begin, because each resolution mutates shared state (canonical field registry, schema registry, identity hashes). Parallel processing would allow semantic duplicates to register independently, defeating the system's purpose. This creates a throughput bottleneck during bulk ingestion.

Embedding model latency. Each field resolution requires embedding computation and similarity search against all canonical fields. This cost compounds with the serial constraint: latency per resolution directly reduces throughput.

Cold-start ordering sensitivity. The first schema registered establishes its field names as canonical names. Idiosyncratic naming in the first schema propagates to all subsequent resolutions. Correctable via manual override or retroactive canonicalization, but initial canonical choices are determined by registration order.

11 Mitigation: Snapshot-Guided Pre-Resolution

The serial bottleneck can be mitigated by introducing a **snapshot-guided pre-resolution** stage before the centralized pipeline (Figure 5).

Approach. Before submitting a schema for registration, the creation process receives a **registry snapshot**—a cached summary of all canonical field names, descriptions, aliases, and schema names. The creation process (whether AI, manual, or import tool) uses this context to align field names with existing canonicals before submission.

Throughput benefits. Pre-resolved schemas arrive with canonical field names and matching descriptions, producing three benefits: (1) reduced state mutations—the canonical field registry is not mutated for pre-resolved fields, eliminating write-ordering constraints; (2) earlier hash convergence—exact hash match succeeds on the first pass, short-circuiting expensive similarity checks; (3) parallelizable residuals—pre-resolved schemas need only read-only registry access and can be processed in parallel, with only genuinely novel schemas requiring serial processing.

Consistency and cold-start. Pre-resolution is advisory: the centralized pipeline remains authoritative and catches any mismatches from stale snapshots. During cold-start, the snapshot refreshes mid-import as early schemas establish canonical fields, reducing ordering sensitivity from "first schema wins" to "first few schemas establish conventions."

12 Conclusion

We presented a two-tier canonicalization system that prevents data structure fragmentation at both schema and field granularity. By matching on field *descriptions* rather than names, the system resolves synonyms and disambiguates homonyms without manual curation. Field-level canonicalization feeds into schema-level deduplication through identity hash recomputation, and schema expansion via field mappers to deprecated schemas enables zero-migration structural growth. The result is a self-organizing namespace that converges toward canonical representations as data is ingested from diverse sources.

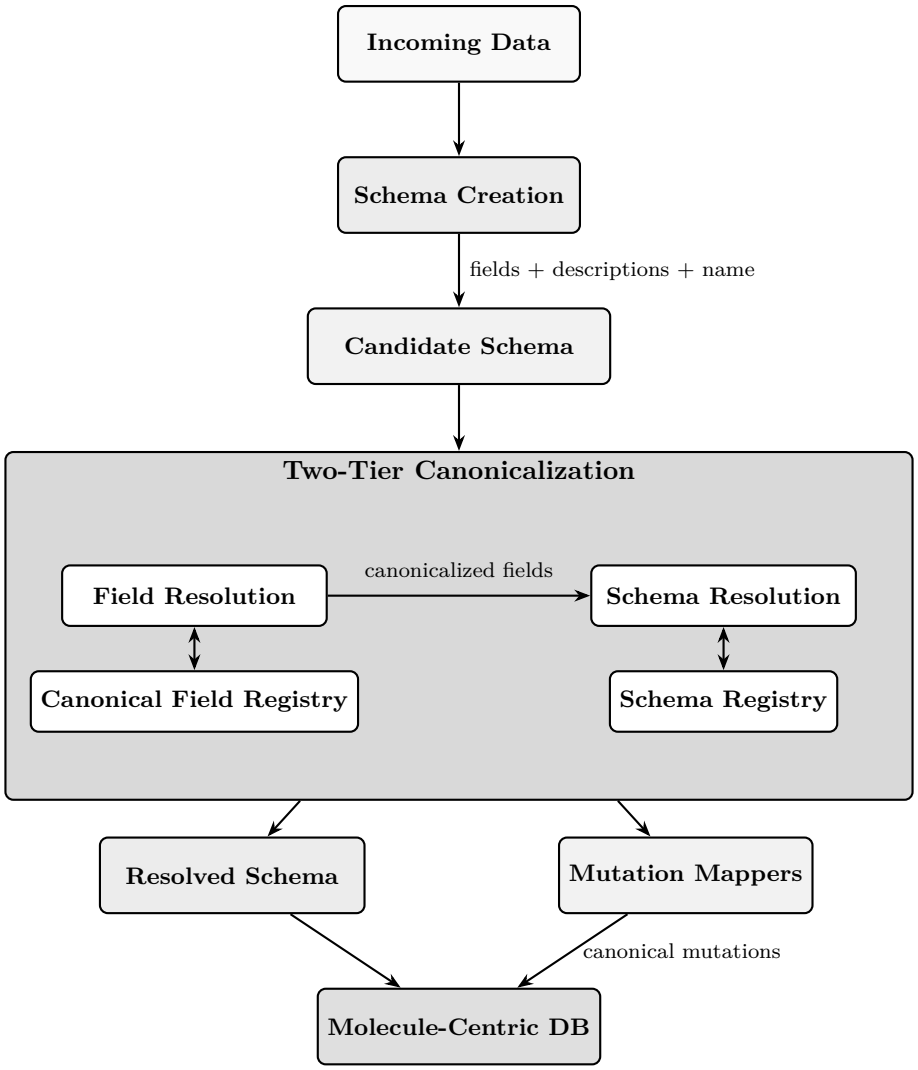

Figure 1: The two-tier canonicalization system sits between schema creation and the database. Tier 1 (field resolution) matches field descriptions against canonical field descriptions in the registry. Tier 2 (schema resolution) deduplicates schemas using the canonicalized field names. When expansion occurs, field mappers to the deprecated schema preserve molecule references with zero data migration.

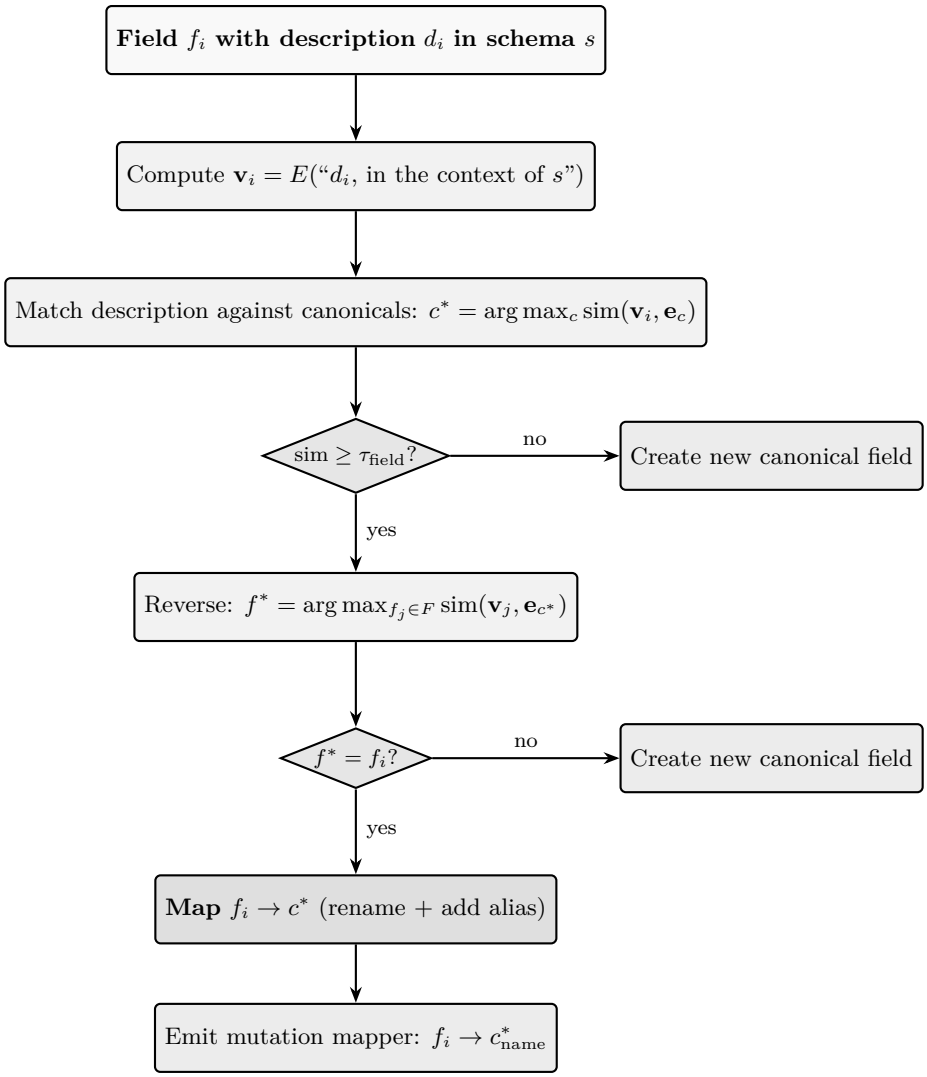

Figure 2: Canonical field resolution algorithm (Tier 1). Each candidate field's text description is embedded with schema context, compared against canonical field descriptions in the registry, and bidirectionally verified. Fields whose descriptions pass both checks are renamed to the canonical field name; fields that fail either check establish new canonical fields.

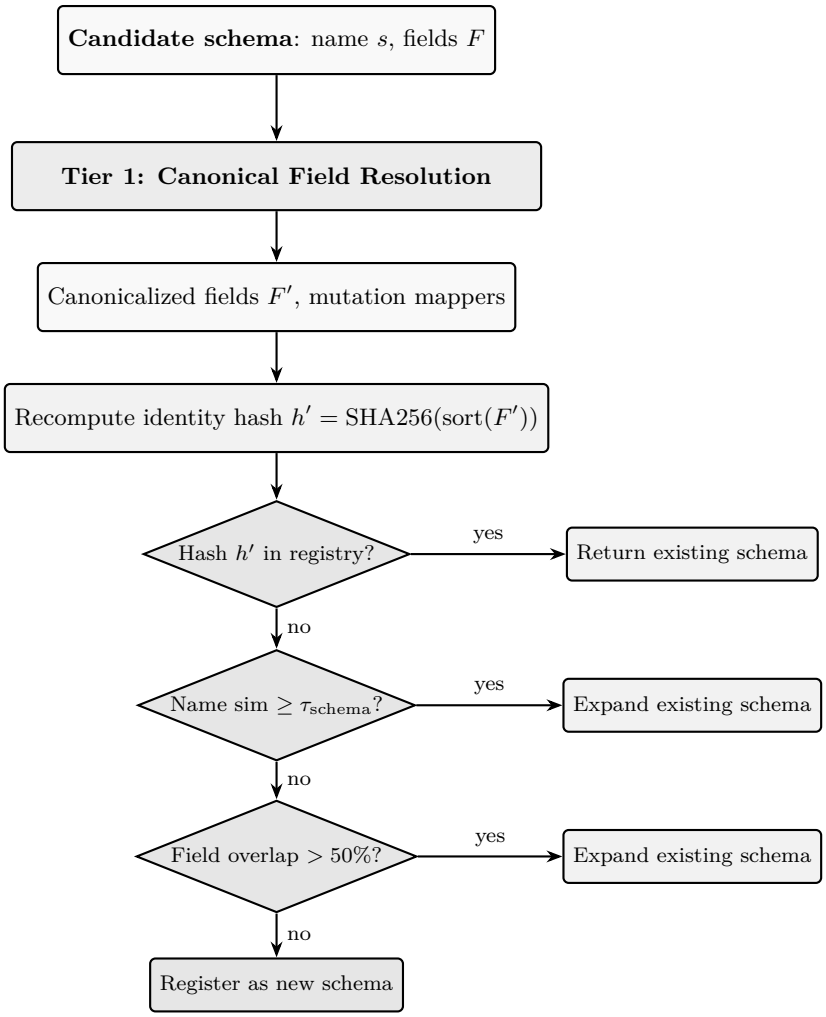

Figure 3: Complete two-tier pipeline. Field canonicalization (Tier 1) runs first, producing canonical field names. The identity hash is recomputed from canonical names. Schema-level deduplication (Tier 2) proceeds through three passes: exact hash match, descriptive name similarity, and field overlap. At any pass, a match triggers schema reuse or expansion via field mappers to the deprecated schema.

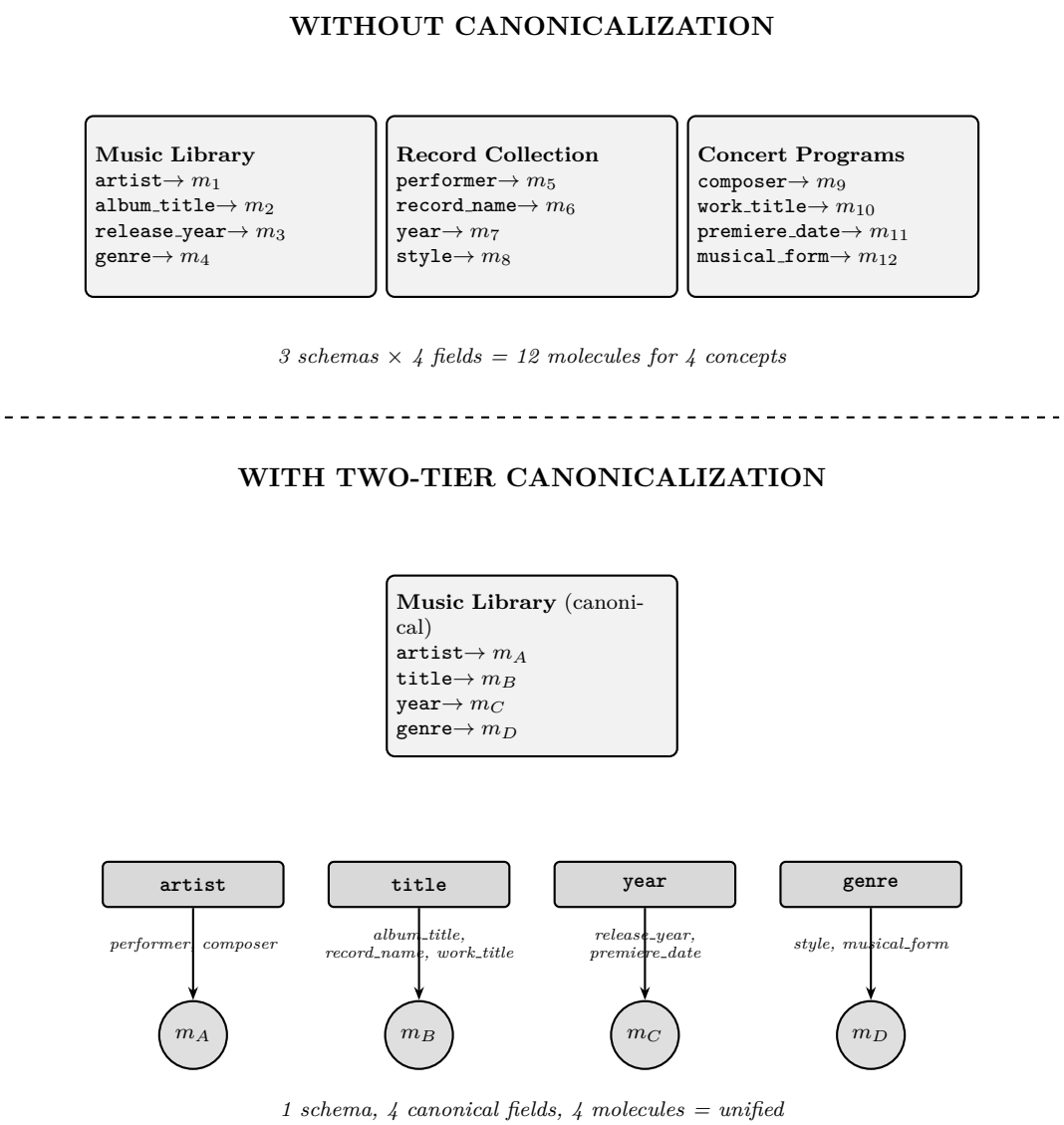

Figure 4: Fragmentation comparison. Without canonicalization (top), three independently inferred schemas create 12 molecules for 4 concepts. With two-tier canonicalization (bottom), all data is accessible through a single schema with 4 canonical fields pointing to 4 molecules. Aliases preserve the mapping from original field names for transparent mutation redirection.

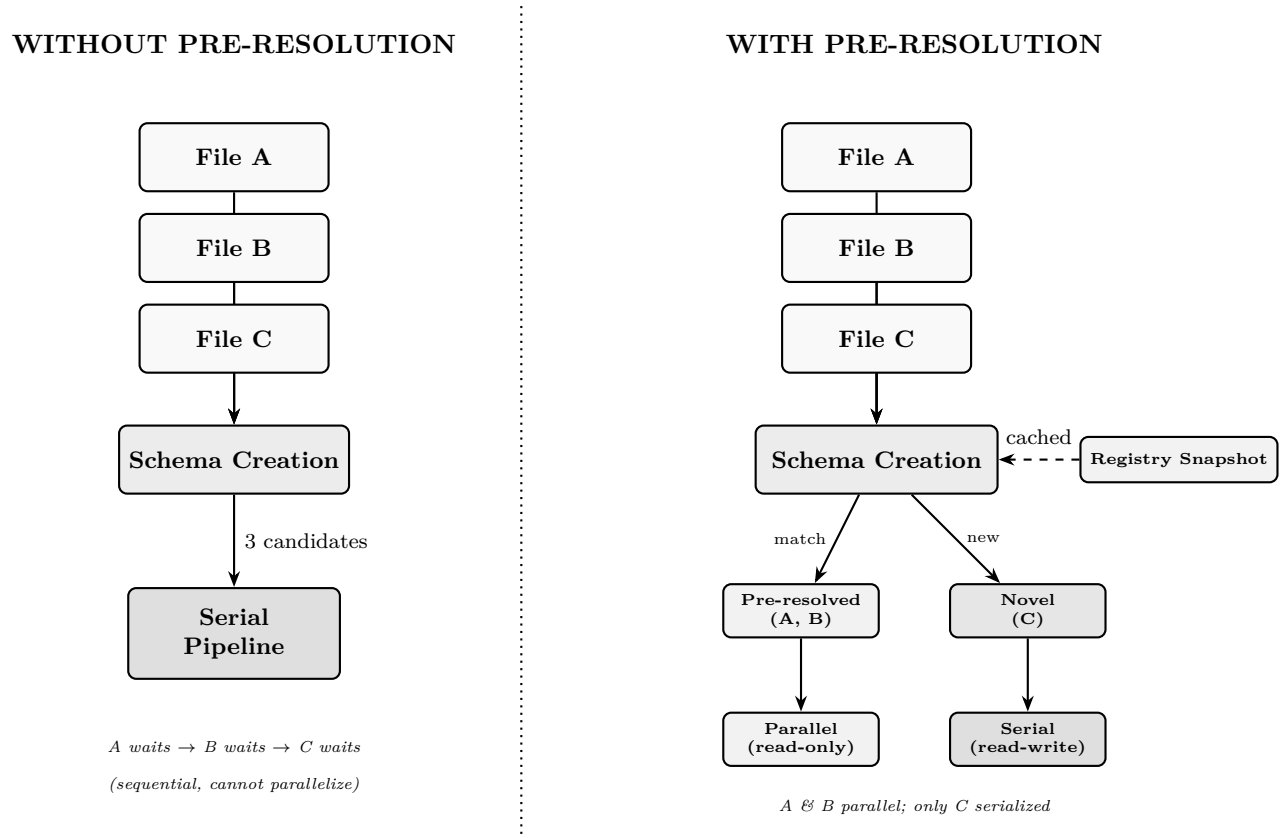

Figure 5: Snapshot-guided pre-resolution. Without pre-resolution (left), all candidates queue for serial processing. With pre-resolution (right), the schema creation process receives a cached registry snapshot and aligns field names with existing canonicals. Pre-resolved schemas are processed in parallel; only genuinely novel schemas require serial processing.